

EPI Quantum Crypto team
Applied cryptography and entropy source assessment

USER GUIDE · VERSION 3.1

libsp80090b: C API, labs and integration code

Building the library, calling its four functions, generating the data and running the labs, integrating a source into a hybrid generator

REFERENCE	NT-02
SUBJECT	libsp80090b: how to use the library and the labs
VERSION	3.1 — September 2026
AUTHORS	EPI Quantum Crypto team
DISTRIBUTION	User guide — public release
STANDARDS	NIST SP 800-90B, SP 800-90C; FIPS 140-3 (IG D.K)
RELATED DOCUMENTS	NT-01 Technical note (method); NT-03 Good-practice guide; French version: NT-02 (fr)
SUPERSEDES	NT-02 v3.0

ABSTRACT

This guide is for C developers, integration engineers and test bench operators. Note NT-01 sets out the NIST SP 800-90B method; this guide gives the operational instructions for **libsp80090b**: building it and its prerequisites (C99, OpenMP, libdivsufsort, libbz2), the structures and the four functions of the C API, data generation and the four labs, and the C code for integration into a hybrid generator: qualification on the target machine, continuous RCT and APT health tests, collection of the DRBG seed. The figures and the known limitations are in the report NT-03.

Keywords: libsp80090b; C API; sp80090b_iid_evaluate_ex; sp80090b_non_iid_evaluate_ex; sp80090b_restart_evaluate; sp80090b_conditioning_evaluate; gen_data.sh; labs; health tests; seed

Contents

1 Overview and directory tree 3

2 Building and prerequisites 3

2.1 Dependencies 3

2.2 Makefile targets 3

3 The C API 4

3.1 Options and result of tests 1 and 2 4

3.2 The four functions 4

3.3 Return codes and warnings 5

4 Data and labs 5

4.1 Generating the data: lab/gen_data.sh 6

4.2 Recording real noise: jitterentropy 6

4.3 Running the four labs 6

5 C integration code 6

5.1 Conformity decision and qualification 7

5.2 Continuous RCT and APT health tests 8

5.3 Start-up and collection of the DRBG seed 9

6 Verifying an installation 9

7 Integration recommendations 10

8 References 10

1 Overview and directory tree

libsp80090b is a C99 port of the NIST tool SP800-90B_EntropyAssessment 1.1.8 [2]. It assesses an **in-memory buffer**: no file to write, no executable to launch, no text output to parse.

Directory tree of SP800-90B_lib

```
SP800-90B_lib/
├── Makefile                  make, make lib, make lab, make check, make docs
├── README.md, README.fr.md   entry point (English, French)
├── include/
│   ├── sp80090b.h            public API: the only header to include
│   └── *.h                   internal headers, one per estimator (ported from the NIST C++
code)
├── src/
│   ├── sp80090b_api.c        the four API functions
│   └── test_*.c              test_iid, test_non_iid, test_restart, test_conditioning,
test_selftest
├── lab/
│   ├── gen_data.sh           generator of the 24 lab datasets
│   ├── lab1.sh .. lab4.sh    the four labs
│   ├── data/                 lab1_iid/, lab2_non_iid/, lab3_restart/, lab4_conditioning/
│   ├── results/              same directories: <case>.res (full output), labN.log
│   ├── lab_*.c               command-line tools compatible with the NIST ea_* tools
│   └── jent_record.c         recorder of jitterentropy raw noise
├── bin/                      the 11 NIST reference datasets
├── selftest/                 copy of the NIST selftest (conformity, section 6)
└── docs/                     NT-01, NT-02, NT-03: sources, style, publications
```

2 Building and prerequisites

2.1 Dependencies

Ubuntu / Debian packages

```
sudo apt-get install -y build-essential libdivsufsort-dev libbz2-dev openssl
```

The library depends on no C++ library, nor on jsoncpp, gmp or mpfr. At run time, it only uses libdivsufsort [8] (suffix array, for the longest repeated substring test), libbz2 (compression statistic), libgomp (OpenMP parallelism), libm and libc. openssl is only used by `gen_data.sh`.

2.2 Makefile targets

Makefile targets

```
make            # static and shared libraries, labs and test programs
make lib        # libsp80090b.a and libsp80090b.so only
make lab        # lab/lab_iid, lab_non_iid, lab_restart, lab_conditioning
make check      # conformity checks (section 6)
make clean      # remove binaries and objects
```

Optimisation and portability. By default, `CFLAGS` is `-std=c99 -O2 -fopenmp -ffloat-store -ffp-contract=fast -march=native`. The `-ffp-contract=fast` option contracts multiply-adds as the NIST C++ compiler does: this is what makes the results bit-for-bit identical to those of the `ea_*` tools built on the same machine. The binary then only runs on equivalent processors. For a portable binary, set the architecture with `make MARCH=-march=x86-64-v2`; the last digits of the results may then differ.

3 The C API

A single header is included, `sp80090b.h`, and linking is done as follows:

Compiling and linking

```
gcc -std=c99 -O2 -fopenmp main.c -I./include libsp80090b.a \
    -ldivsufsort -ldivsufsort64 -lbz2 -lm -o my_app
```

3.1 Options and result of tests 1 and 2

sp80090b.h: options and result

```
typedef struct {
    int bits_per_symbol;       /* 1..8, or 0 to infer from the data */
    bool initial_entropy;      /* true: -i, H_I of raw noise; false: -c, h' of conditioned data
*/
    bool truncate_bitstring;   /* true: -t, H_bitstring on the first 1,000,000 bits */
    int verbose;               /* 0 silent; 1..3 as the NIST tool, on stdout */
    const uint64_t *seed;      /* IID only: 4 words for reproducible permutations; NULL =
/dev/urandom */
} sp80090b_options_t;

typedef struct {
    double h_original;         /* min-entropy of the samples, bits per sample */
    double h_bitstring;        /* min-entropy of the bit string, bits per bit; h' with -c */
    double h_assessed;         /* min(H_original, n x H_bitstring); h' with -c */
    bool passed_iid;           /* IID: chi-square AND LRS AND permutation tests passed */
    double p_value;            /* IID: smaller chi-square p-value */
    int status;                /* return code of the call */
    char error_msg[256];       /* error text, or last warning text */
    unsigned int warnings;     /* SP80090B_WARN_* flags */
    long n_samples;            /* samples assessed */
    int bits_per_symbol;       /* width used (inferred when 0 was given) */
    int alphabet_size;         /* distinct symbols observed */
    bool passed_chi_square;    /* IID: 5.2.1-5.2.4 */
    bool passed_lrs;           /* IID: 5.2.5 */
    bool passed_permutation;   /* IID: 5.1 */
    int perm_counts[19][3];    /* IID: C[i][0] (>), C[i][1] (=), C[i][2] (<) for the 19
statistics */
} sp80090b_result_t;
```

3.2 The four functions

Test 1: IID track (§5).

Test 1

```
int sp80090b_iid_evaluate_ex(const uint8_t *data, size_t len,
                             const sp80090b_options_t *opt, sp80090b_result_t *result);
```

> **Warning.** `result->h_assessed` is usable **only if** `result->passed_iid` **is true**. Otherwise, the source must be assessed through the non-IID track.

Test 2: non-IID track (§6.3). Computes the ten estimators and returns H_I in `result->h_assessed`.

Test 2

```
int sp80090b_non_iid_evaluate_ex(const uint8_t *data, size_t len,
                                 const sp80090b_options_t *opt, sp80090b_result_t *result);
```

Test 3: restart (§3.1.4).

Test 3

```
int sp80090b_restart_evaluate(const uint8_t *data, size_t len, int bits_per_symbol,
                              double h_i, bool iid, unsigned long rounds,
                              const uint64_t *seed, int verbose,
                              sp80090b_restart_result_t *result);
```

- `data`: 1,000 × 1,000 sample matrix stored by rows, restart i at offset 1,000 × i; `len` is 1,000,000.
- `h_i`: H_I from test 1 or test 2; `iid` selects the IID track on the rows and columns.
- `rounds`: draws of the cutoff simulation; 0 for the NIST 5,000,000, a lower value is refused.
- Result: `sanity_passed` (§3.1.4.3), `validation_passed` (§3.1.4.2), `x_max`, `x_cutoff`, `h_r`, `h_c` and the validated entropy `h_restart` = min(H_r, H_c, H_I).

Test 4: conditioning (§3.1.5).

Test 4

```
int sp80090b_conditioning_evaluate(bool vetted, unsigned int n_in, unsigned int n_out,
                                   unsigned int nw, long double h_in, long double h_prime,
                                   sp80090b_conditioning_result_t *result);
```

- `vetted`: true for a vetted function (SHA-2, SHA-3, HMAC, CMAC, CBC-MAC); otherwise `h_prime` $\in]0, 1]$ is the entropy per bit of the output, measured by the non-IID track in `-c` mode.
- `h_in`: total input entropy, number of samples × H.
- Result: `h_out`, `epsilon_log2` and `full_entropy_candidate`, true if the function is vetted, $h_{out} > 0.999\ n_{out}$ and $h_{in} \geq n_{out} + 64$ (FIPS 140-3 IG D.K). The binding condition is the last one: with a vetted function and a 64-bit margin, the entropy deficit per bit is below 2^{-32}.

3.3 Return codes and warnings

Table 1. Return codes.

Code	Value	Meaning
`SP80090B_OK`	0	Assessment done; then read `warnings`
`SP80090B_EINVAL`	−1	Invalid argument: null pointer, fewer than 2 samples, width outside [0, 8], restart or conditioning parameters
`SP80090B_EDATA`	−2	Data wider than `bits_per_symbol`, or out of memory while reading
`SP80090B_EALPHABET`	−3	**Dead source**: a single symbol in the whole buffer
`SP80090B_EINTERNAL`	−4	Allocation, seeding (`/dev/urandom`) or numerical failure

Table 2. Warning flags (warnings field).

Flag	Mask	Meaning
`SP80090B_WARN_FEW_SAMPLES`	0x1	Fewer than 1,000,000 samples (§3.1.1)
`SP80090B_WARN_NARROW_DATA`	0x2	Symbols narrower than the declared width
`SP80090B_WARN_TRUNCATED`	0x4	$H_{bitstring}$ computed on the first million bits only (`-t`)

A warning is not an error, but it invalidates a result meant to qualify a source.

4 Data and labs

The NIST tools and the library expect a binary file with **one sample per byte**: for a 1-bit source, each byte is 0 or 1.

4.1 Generating the data: lab/gen_data.sh

`gen_data.sh` builds 24 datasets in 7 groups: **simulated** physical sources (thermal noise, ring oscillator…) drawn from a seeded model, design defects, environmental effects, failures, DRBG outputs taken from the system (`/dev/urandom`, `openssl`) and restart datasets. Each file is copied into the directory of every lab that reads it.

Generating the data

```
lab/gen_data.sh all --seed 90       # the 24 datasets, fixed seed
lab/gen_data.sh thermal --sd 5      # one dataset, with a model parameter changed
lab/gen_data.sh --list              # groups, datasets and what each one shows
```

4.2 Recording real noise: jitterentropy

`lab/jent_record` records the raw CPU jitter noise measured by jitterentropy [9], built at `-O0` like the library itself; the variable `JENT_SRC` gives its sources.

Recording jitterentropy raw noise

```
make lab/jent_record
lab/jent_record lab/data/lab2_non_iid/jent-raw.bin          # 1,000,000 samples (tests 1 and 2)
lab/jent_record -r lab/data/lab3_restart/jent-restart.bin   # 1,000 restarts x 1,000 (test 3)
```

4.3 Running the four labs

Each lab walks through the files of its data directory, writes the full output to `lab/results/<lab>/<case>.res` and prints one line per case. The labs are run one at a time, in this order:

Running the labs

```
lab/lab2.sh     # non-IID track on lab2_non_iid/: H_I, needed by labs 3 and 4
lab/lab3.sh     # restart test on lab3_restart/
lab/lab4.sh     # conditioning cases
lab/lab1.sh     # IID track on lab1_iid/: several hours if sources are not IID
```

Table 3. The four labs.

Lab	Program	Data	Time per file of 10^6 samples
1	`test_iid -vv`	`lab1_iid/`	Under a minute if IID; 40 minutes to 2 hours otherwise
2	`test_non_iid -vv`	`lab2_non_iid/`	5 seconds to 1 minute
3	`test_restart -v`	`lab3_restart/`	40 seconds to 1 minute 30
4	`test_conditioning`	`lab4_conditioning/`	Under a second per case

A single file can also be run on its own, from `lab/`, for example to resume an interrupted lab: `../test_iid -vv data/lab1_iid/drift.bin 8 > results/lab1_iid/drift.res 2>&1`. The `lab/lab_*` programs also take the command line of the NIST `ea_*` tools, to compare a result.

5 C integration code

This section gives example C code for a hybrid generator (SP 800-90C): conformity decision at qualification, then health tests and seed collection in operation.

5.1 Conformity decision and qualification

The function applies the criteria of NT-01: H_I through the non-IID track, full restart test, then comparison with the claim of the model.

Conformity decision for a source

```
#include <stdbool.h>
#include <stdint.h>
#include "sp80090b.h"

typedef enum {
    ENTROPY_PASSED            =  0,   /* conforming source */
    ERR_INSUFFICIENT_SAMPLES  = -1,   /* fewer than 1,000,000 samples */
    ERR_SOURCE_STUCK          = -2,   /* a single symbol: dead source */
    ERR_RESTART_SANITY_FAILED = -3,   /* X_max > X_cutoff (3.1.4.3) */
    ERR_RESTART_ENTROPY_DROP  = -4,   /* min(H_r, H_c) < H_I / 2 (3.1.4.2) */
    ERR_ENTROPY_BELOW_CLAIM   = -5,   /* validated H below the claim */
    ERR_EVALUATION            = -6    /* argument, data or memory error */
} entropy_decision_t;

/*
 * raw     : at least 1,000,000 consecutive raw samples (steady state)
 * rows    : 1000 restarts x 1000 samples, restart i at offset 1000 * i
 * h_claim : entropy per sample claimed by the model of the source
 * h_out   : credited entropy, min(H_r, H_c, H_I, h_claim)
 */
entropy_decision_t verify_entropy_compliance(const uint8_t *raw, size_t len,
                                             const uint8_t *rows, int bits,
                                             double h_claim, double *h_out)
{
    sp80090b_options_t opt = sp80090b_default_options();
    sp80090b_result_t res;
    sp80090b_restart_result_t rst;
    double h_i, h_measured;
    int status;

    if (!raw || !rows || !h_out)
        return ERR_EVALUATION;

    /* Steps 1-2: H_I through the non-IID track */
    opt.bits_per_symbol = bits;
    status = sp80090b_non_iid_evaluate_ex(raw, len, &opt, &res);
    if (status == SP80090B_EALPHABET)
        return ERR_SOURCE_STUCK;
    if (status != SP80090B_OK)
        return ERR_EVALUATION;
    if (res.warnings & SP80090B_WARN_FEW_SAMPLES)
        return ERR_INSUFFICIENT_SAMPLES;
    if (res.warnings != 0)                    /* any warning invalidates a qualification */
        return ERR_EVALUATION;

    /* SP 800-90B: H_I includes the submitter's claim */
    h_i = res.h_assessed < h_claim ? res.h_assessed : h_claim;

    /* Steps 3-4: restart test, sanity check then H_r and H_c */
    status = sp80090b_restart_evaluate(rows, (size_t)SP80090B_RESTART_ROWS *
SP80090B_RESTART_COLS,
                                       bits, h_i, false, 0, NULL, 0, &rst);
    if (status != SP80090B_OK)
        return ERR_EVALUATION;
    if (!rst.sanity_passed)
        return ERR_RESTART_SANITY_FAILED;
    if (!rst.validation_passed)
        return ERR_RESTART_ENTROPY_DROP;

    /* Step 5: credit the validated entropy; flag a measurement below the claim */
    *h_out = rst.h_restart;                     /* = min(H_r, H_c, H_I) <= h_claim */
    h_measured = res.h_assessed;
```

Conformity decision for a source

```
    if (rst.h_r < h_measured) h_measured = rst.h_r;
    if (rst.h_c < h_measured) h_measured = rst.h_c;
    return h_measured >= h_claim ? ENTROPY_PASSED : ERR_ENTROPY_BELOW_CLAIM;
}
```

5.2 Continuous RCT and APT health tests

This light module runs in the generator on every sample produced. Its cutoffs are computed once, from the credited entropy H (α = 2^{-20}); they match those of NT-01.

Continuous health tests (SP 800-90B 4.4)

```
#include <math.h>
#include <stdbool.h>
#include <stdint.h>

typedef struct {
    unsigned int rct_c;     /* RCT cutoff */
    unsigned int apt_c;     /* APT cutoff */
    unsigned int apt_w;     /* APT window: 1024 for a binary source, 512 otherwise (4.4.2) */
    uint8_t rct_last;       /* last value seen by the RCT */
    uint8_t apt_base;       /* value tracked in the current APT window */
    unsigned int rct_cnt;   /* RCT counter */
    unsigned int apt_cnt;   /* APT counter */
    unsigned int apt_idx;   /* position in the APT window */
} health_ctx_t;

/* APT cutoff: smallest C such that P(X >= C) <= 2^-20, X ~ Binomial(w, 2^-h) */
static unsigned int compute_apt_cutoff(unsigned int w, double h)
{
    double p = exp2(-h), tail = 0.0;

    for (unsigned int c = w; c > 0; c--) {
        tail += exp(lgamma(w + 1.0) - lgamma(c + 1.0) - lgamma(w - c + 1.0)
                    + c * log(p) + (w - c) * log1p(-p));
        if (tail > 0x1p-20)
            return c + 1;
    }
    return 1;
}

/* Cutoffs from the credited entropy H */
void health_init(health_ctx_t *ctx, double h_credited, bool is_binary)
{
    ctx->rct_c = 1 + (unsigned int)ceil(20.0 / h_credited);
    ctx->apt_w = is_binary ? 1024 : 512;
    ctx->apt_c = compute_apt_cutoff(ctx->apt_w, h_credited);
    ctx->rct_cnt = 0;
    ctx->apt_idx = 0;
}

/* 0 if the sample passes, -1 if the source must be stopped */
int health_check_sample(health_ctx_t *ctx, uint8_t sample)
{
    if (ctx->rct_cnt > 0 && sample == ctx->rct_last) {
        if (++ctx->rct_cnt >= ctx->rct_c)
            return -1;                          /* RCT: abnormal repetition */
    } else {
        ctx->rct_last = sample;
        ctx->rct_cnt = 1;
    }
    if (ctx->apt_idx == 0) {                    /* start of an APT window */
        ctx->apt_base = sample;
        ctx->apt_cnt = 1;
    } else if (sample == ctx->apt_base && ++ctx->apt_cnt >= ctx->apt_c) {
```

Continuous health tests (SP 800-90B 4.4)

```
        return -1;                              /* APT: abnormal proportion */
    }
    ctx->apt_idx = (ctx->apt_idx + 1) % ctx->apt_w;
    return 0;
}
```

5.3 Start-up and collection of the DRBG seed

Start-up and seed collection

```
#include <stddef.h>
#include <stdint.h>

uint8_t read_hardware_noise_sample(void);   /* acquisition, specific to the source */

/* Start-up tests: at least 1,024 consecutive samples must pass (4.3) */
int health_startup(health_ctx_t *ctx)
{
    for (unsigned int i = 0; i < 1024; i++)
        if (health_check_sample(ctx, read_hardware_noise_sample()) != 0)
            return -1;
    return 0;
}

/* Wipe that the compiler cannot remove (use explicit_bzero or memset_s when available) */
static void secure_wipe(void *p, size_t n)
{
    volatile uint8_t *v = (volatile uint8_t *)p;
    while (n--)
        *v++ = 0;
}

/* Seed: N = ceil(E / H) raw samples, all tested; at the first failure, the seed is wiped */
int collect_drbg_seed(health_ctx_t *ctx, uint8_t *seed_buf, size_t n_samples)
{
    for (size_t i = 0; i < n_samples; i++) {
        uint8_t raw = read_hardware_noise_sample();

        if (health_check_sample(ctx, raw) != 0) {
            secure_wipe(seed_buf, n_samples);   /* discard the current seed */
            return -1;
        }
        seed_buf[i] = raw;
    }
    return 0;                                   /* entropy input of the DRBG */
}
```

The N seed bytes form the entropy input of the DRBG, directly or after conditioning. For a 256-bit strength, N = ⌈384 / H⌉ at instantiation and ⌈256 / H⌉ at reseed (NT-01).

6 Verifying an installation

Repeat these checks after any change of compiler, compiler options or machine:

Verification

```
cd Entropy/SP800-90B_lib
make check
```

`make check` runs two checks:

1. `selftest/selftest`, a copy of the selftest of the NIST tool: it assesses the eleven reference datasets of `bin/` with `test_non_iid` and compares every intermediate value with the NIST outputs in `selftest/refdata/`. The verification passes if no gap exceeds 10^{-10}; the largest gap observed is 7.2×10^{-14}.
2. `test_selftest`: NIST values on reference datasets, API contract (invalid arguments, warnings, return codes), conditioning and reproducibility with a fixed seed.

7 Integration recommendations

1. **Always check the return code** (`SP80090B_OK`), then the `warnings` field.
2. **Never use `h_assessed` from the IID track** if `passed_iid` is false.
3. **Assess raw noise only**, never a DRBG or hash output: it gets close to 8 bits whatever its seed.
4. **Size the seed** with $N = \lceil E / H \rceil$: E = 384 bits at instantiation, 256 bits at reseed.
5. **Run the RCT and APT health tests** on every raw sample, before conditioning and the DRBG, and the start-up tests on 1,024 samples.
6. **Provide a 64-bit margin** of input entropy for any conditioning function aiming at full entropy: $h_{in} \geq n_{out} + 64$.
7. **Limit the OpenMP threads** (`OMP_NUM_THREADS`) when several assessments run in parallel, and allow about 250 MB of memory per assessment of one million samples.

8 References

[2] National Institute of Standards and Technology. *SP800-90B_EntropyAssessment*. Source code, github.com/usnistgov/SP800-90B_EntropyAssessment.

[8] Y. Mori. *libdivsufsort — suffix-sorting library*. Source code, version 2.0.

[9] S. Müller. *Jitter RNG (jitterentropy)*. Documentation and source code, version 3.7.1, 2026.

Reference numbers are those of note NT-01.